

Anteraja Instant — Basis Data

Perancangan Database Relasional untuk Delivery Integrity

Produk	Anteraja — Satria Rapid Field Dispatch
DBMS	PostgreSQL 14+ dengan PostGIS & pgcrypto
Fokus	Integritas pengiriman: geofence, POD, PIN, titik temu, audit klaim
Skala	18 tabel · 2 view · 2 function · 1 trigger shared
Referensi	docs/PRD-anteraja-instant.md & docs/frd/FRD-01..05
Repositori	branch 6-db · docs/db/

Ringkasan

Dokumen ini memuat hasil perancangan basis data relasional untuk modul **Anteraja Instant**. Skema diturunkan dari PRD dan lima FRD, lalu dinormalisasi menjadi tabel yang mendukung setiap kebutuhan verifikasi pengiriman **last-mile**: kunci geofence yang dihitung di server, bukti pengiriman ber-geotag, verifikasi PIN per segmen, matchmaking titik temu, hingga audit trail investigasi klaim.

Seluruh objek SQL tersedia pada berkas `schema.sql`, data contoh pada `seed.sql`, dan query verifikasi pada `queries.sql`. Diagram ERD disajikan pada `erd.webp`, dan detail tiap tabel dibahas pada bab berikut.

Integritas Objektif Koordinat disimpan sebagai <code>geography(Point, 4326)</code> ; jarak dihitung dengan <code>ST_Distance</code> ; keputusan lulus/tidak selalu di server.	Dapat Diaudit Setiap peristiwa & keputusan tercatat (<code>delivery_events</code> , <code>admin_actions</code>), dengan view audit trail dan skor anomali per pengiriman.
--	---

Daftar Isi

1. Konvensi & Normalisasi
2. Diagram ERD
3. Kamus Tabel
4. Skema SQL
5. Data Contoh
6. Query Verifikasi

7. Pemetaan FRD → Tabel
8. Cara Menjalankan & Tautan Branch
9. Lampiran — Bukti Branch

1. Konvensi & Normalisasi

Konvensi Penamaan

- **Tabel:** snake_case, bentuk jamak (shipments, delivery_events).
- **Kolom:** snake_case; kunci asing berpola <entitas>_id (courier_id).
- **Kunci utama:** id uuid dengan gen_random_uuid().
- **Kolom audit:** created_at / updated_at (timestampz), updated_at dipelihara trigger fn_touch_updated_at.
- **Status/tipe:** dibatasi CHECK (portabel, tanpa enum type).
- **Koordinat:** geography(Point, 4326) (WGS84). **Jarak:** integer meter.

Normalisasi

Skema berada pada **3NF**:

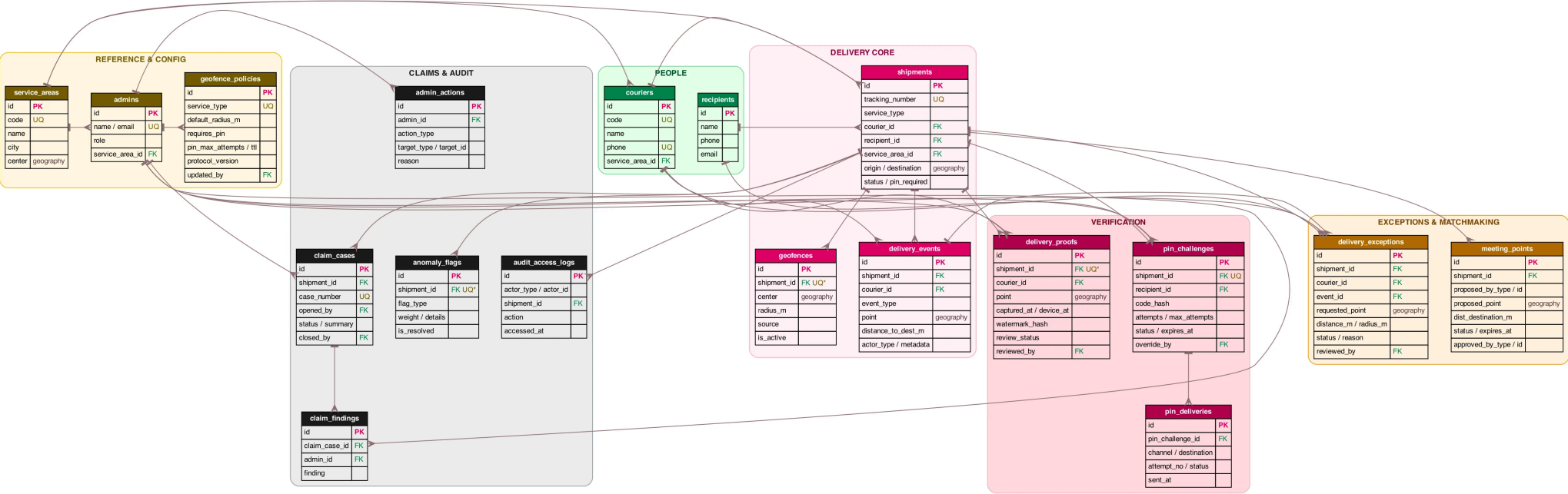
Bentuk	Penerapan
1NF	Kolom atomik, tanpa grup berulang. Kanal PIN (pin_deliveries), temuan klaim (claim_findings), dan peristiwa (delivery_events) dipisahkan ke tabel sendiri.
2NF	Setiap tabel ber-PK tunggal id uuid; seluruh atribut bergantung penuh pada PK.
3NF	Tanpa ketergantungan transitif: kebijakan radius/PIN ke geofence_policies, wilayah ke service_areas, identitas admin ke admins (tidak diduplikasi).

Aturan bisnis dijaga di lapisan basis data melalui **partial unique index**:

- satu geofence aktif per pengiriman (ux_geofences_one_active);
- satu POD valid per pengiriman (ux_proofs_one_valid);
- satu titik temu final per pengiriman (ux_meeting_points_one_final);
- satu pengajuan pengecualian pending per pengiriman (ux_exceptions_one_pending).

2. Diagram ERD (crow's foot)

Anteraja Instant – Delivery Integrity Database
PostgreSQL + PostGIS · branch 6-db · crow's-foot notation



3. Kamus Tabel

3.1 Reference & Configuration

- `service_areas` — wilayah/hub. `id` PK, `code` UQ, `name`, `city`, `center`, `is_active`.
- `admins` — aktor Admin/CS. `id` PK, `email` UQ, `role` (superadmin/ops_admin/cs_agent), `service_area_id` FK.
- `geofence_policies` — kebijakan per segmen. `service_type` UQ, `default_radius_m` (instant 30 / same_day 50 / regular 100), `requires_pin`, `pin_max_attempts`, `pin_ttl_minutes`, `pin_max_resends`, `protocol_version`, `updated_by` FK.

3.2 People

- `couriers` — kurir SATRIA. `code` UQ, `phone` UQ, `service_area_id` FK.
- `recipients` — penerima/pembeli. `name`, `phone`, `email` (tujuan PIN).

3.3 Delivery Core

- `shipments` — inti pengiriman. `tracking_number` UQ, `service_type`, FK `courier_id`/`recipient_id`/`service_area_id`, `origin/destination geography`, `status`, `pin_required`, `cod_amount`, `delivered_at`.
- `geofences` — pusat & radius. `center geography`, `radius_m`, `source` (destination/meeting_point), `is_active`; index GiST.
- `delivery_events` — log peristiwa. `event_type`, `point`, `distance_to_destination_m`, `actor_type`, `metadata jsonb`.

3.4 Verification

- `delivery_proofs` — POD ber-geotag. `point`, `distance_to_destination_m`, `captured_at`, `device_captured_at`, `watermark_hash`, `review_status`, `reviewed_by` FK.
- `pin_challenges` — satu per pengiriman. `code_hash` (hash, bukan PIN), `attempts`, `max_attempts`, `resend_count`, `status`, `expires_at`, `override_by` FK.
- `pin_deliveries` — riwayat kirim PIN. `channel`, `destination`, `attempt_no`, `status`, `sent_at`.

3.5 Exceptions & Matchmaking

- `delivery_exceptions` — penyelesaian diluar radius. `requested_point`, `distance_m`, `radius_m`, `reason`, `status`, `reviewed_by` FK.
- `meeting_points` — usulan & persetujuan titik temu. `proposed_by_type/id`, `proposed_point`, `status`, `approved_by_type/id`, `expires_at`.

3.6 Claims & Audit

- `claim_cases` — kasus klaim. `case_number` UQ, `opened_by` FK, `status`, `resolution`, `closed_by` FK.
- `claim_findings` — temuan investigasi. `claim_case_id` FK, `admin_id` FK, `finding`.
- `anomaly_flags` — penanda anomali. `flag_type`, `weight`, `details`, `is_resolved`; unique (`shipment_id`, `flag_type`).
- `audit_access_logs` — log akses audit. `actor_type/id`, `shipment_id`, `action`, `accessed_at`.
- `admin_actions` — audit keputusan admin. `action_type`, `target_type/id`, `reason`.

4. Skema SQL

Potongan dari `schema.sql` (DDL lengkap ada pada berkas di branch `6-db`).

4.1 Ekstensi & Tabel Inti

```
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE shipments (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tracking_number   text NOT NULL UNIQUE,
  service_type      text NOT NULL
                    CHECK (service_type IN ('instant','same_day','regular')),
  courier_id        uuid REFERENCES couriers(id) ON DELETE SET NULL,
  recipient_id      uuid NOT NULL REFERENCES recipients(id),
  service_area_id   uuid NOT NULL REFERENCES service_areas(id),
  origin            geography(Point,4326) NOT NULL,
  destination       geography(Point,4326) NOT NULL,
  status            text NOT NULL DEFAULT 'pending'
                    CHECK (status IN ('pending','picked_up','in_transit','delivered','failed')),
  pin_required      boolean NOT NULL DEFAULT false,
  cod_amount        integer NOT NULL DEFAULT 0 CHECK (cod_amount >= 0),
  delivered_at      timestampz,
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now()
);

CREATE TABLE geofences (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  shipment_id       uuid NOT NULL REFERENCES shipments(id) ON DELETE CASCADE,
  center            geography(Point,4326) NOT NULL,
  radius_m          integer NOT NULL CHECK (radius_m > 0),
  source            text NOT NULL DEFAULT 'destination'
                    CHECK (source IN ('destination','meeting_point')),
  is_active         boolean NOT NULL DEFAULT true,
  created_at        timestampz NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX ux_geofences_one_active
  ON geofences (shipment_id) WHERE is_active;
CREATE INDEX ix_geofences_center_gist ON geofences USING gist (center);
```

4.2 Fungsi Evaluasi Geofence (server-side)

```
CREATE OR REPLACE FUNCTION fn_evaluate_geofence(
  p_shipment_id uuid, p_latitude double precision, p_longitude double precision
) RETURNS TABLE (distance_m integer, radius_m integer, inside boolean)
LANGUAGE sql STABLE AS $$
SELECT
  round(ST_Distance(g.center,
    ST_SetSRID(ST_MakePoint(p_longitude, p_latitude), 4326)::geography)::integer,
  g.radius_m,
  ST_Distance(g.center,
    ST_SetSRID(ST_MakePoint(p_longitude, p_latitude), 4326)::geography) <= g.radius_m
FROM geofences g
WHERE g.shipment_id = p_shipment_id AND g.is_active
ORDER BY g.created_at DESC LIMIT 1;
$$;
```

4.3 View Skor Anomali

```
CREATE OR REPLACE VIEW v_shipment_anomaly_score AS
SELECT s.id AS shipment_id, s.tracking_number,
```

```
coalesce(sum(a.weight) FILTER (WHERE NOT a.is_resolved), 0)::numeric(6,2) AS anomaly_score,  
count(a.id) FILTER (WHERE NOT a.is_resolved) AS open_flag_count,  
coalesce(sum(a.weight) FILTER (WHERE NOT a.is_resolved), 0) >= 2.00 AS needs_review,  
coalesce(array_agg(a.flag_type ORDER BY a.flag_type)  
  FILTER (WHERE NOT a.is_resolved AND a.flag_type IS NOT NULL), '{}') AS flags  
FROM shipments s  
LEFT JOIN anomaly_flags a ON a.shipment_id = s.id  
GROUP BY s.id, s.tracking_number;
```

5. Data Contoh

seed.sql memuat 15 pengiriman (selaras docs/data/shipments-couriers-coordinates.csv) dengan skenario yang mencakup seluruh FRD.

5.1 Cuplikan Seed

```
INSERT INTO shipments (id, tracking_number, service_type, courier_id, ...)
SELECT v.id::uuid, v.tracking, v.service_type, c.id, ...
FROM (VALUES
  (...001, 'AJ2509000001', 'instant', '...STR-001', '...rec01', '...JKS',
   106.827153, -6.175392, 'Jl. Senopati No. 12, ...', 'delivered', true, 0,
   '2026-09-22T09:14:00+07', '2026-09-22T08:10:00+07'),
  (...005, 'AJ2509000005', 'instant', '...STR-005', '...rec05', '...JKT',
   106.854000, -6.229000, 'Jl. Letjen Suprpto No. 21, ...', 'in_transit', true, 0,
   NULL, '2026-09-22T13:10:00+07')
) AS v(...) JOIN couriers c ON c.id = v.courier_id::uuid ...;
```

5.2 Skenario yang Dicapuk

- Pengiriman normal di dalam radius (instant/same_day/regular).
- Geofence gagal: ...0005 (912 m), ...0006 (556 m), ...0012 (2170 m).
- PIN: verified, locked setelah 3x salah (...0007), expired (...0008), pending, serta override admin (...0014).
- POD needs_review: di luar geofence (...0006) & selisih waktu perangkat (...0010, ±170 menit).
- Pengecualian: pending (...0005), approved (...0006, ...0014).
- Titik temu: final approved (...0006), proposed (...0005).
- Klaim: investigating (...0006) dan closed (...0007) + temuan.
- Anomali: **4 pengiriman perlu tinjauan** (...0005, ...0006, ...0007, ...0010) — cocok dengan dashboard UI.

5.3 Hasil Skor Anomali

tracking_number	anomaly_score	flags	needs_review
AJ2509000006	3.00	{exception_used, out_of_radius, pod_needs_review}	t
AJ2509000005	2.00	{out_of_radius}	t
AJ2509000007	2.00	{repeated_pin_failure}	t
AJ2509000010	2.00	{device_time_mismatch}	t
AJ2509000014	0.50	{exception_used}	f

6. Query Verifikasi

Potongan dari `queries.sql`.

6.1 Keputusan Geofence per Pengiriman (FRD-01)

```
SELECT s.tracking_number, e.event_type, e.distance_to_destination_m, g.radius_m,
       (e.distance_to_destination_m <= g.radius_m) AS inside
FROM delivery_events e
JOIN shipments s ON s.id = e.shipment_id
JOIN geofences g ON g.shipment_id = s.id AND g.is_active
WHERE e.event_type = 'geofence_check'
ORDER BY s.tracking_number;
```

tracking_number	dist	radius_m	inside
AJ2509000001	1	30	t
AJ2509000003	40	50	t
AJ2509000005	912	30	f
AJ2509000006	556	50	f
AJ2509000009	100	100	t (tepat di batas)
AJ2509000012	2170	50	f
AJ2509000014	260	30	f (exception approved)

6.2 PIN Tanpa Verifikasi Tidak Boleh delivered (FR-03-07)

```
SELECT s.tracking_number, s.service_type, s.status, pc.status AS pin_status
FROM shipments s
JOIN pin_challenges pc ON pc.shipment_id = s.id
WHERE s.status = 'delivered' AND pc.status NOT IN ('verified', 'override');
-- expected: 0 rows
```

6.3 Invarian Basis Data

```
-- tidak boleh ada > 1 geofence aktif / POD valid / titik temu final per pengiriman
SELECT 'active geofences' AS invariant, count(*) AS violations
FROM (SELECT shipment_id FROM geofences WHERE is_active
      GROUP BY shipment_id HAVING count(*) > 1) t
UNION ALL
SELECT 'valid PODs', count(*) FROM (SELECT shipment_id FROM delivery_proofs
      WHERE review_status = 'valid' GROUP BY shipment_id HAVING count(*) > 1) t
UNION ALL
SELECT 'final meeting points', count(*) FROM (SELECT shipment_id FROM meeting_points
      WHERE status IN ('approved', 'admin_set') GROUP BY shipment_id HAVING count(*) > 1) t;
```

7. Pemetaan FRD → Tabel

FRD	Tabel utama	Pendukung
01	geofences, delivery_events, shipments	delivery_exceptions, geofence_policies, fn_evaluate_geofence()
02	delivery_proofs	anomaly_flags (pod_needs_review)
03	pin_challenges	pin_deliveries, geofence_policies, admin_actions
04	meeting_points	delivery_events, geofences (source = meeting_point)
05	v_shipment_audit_trail, claim_cases, anomaly_flags	claim_findings, audit_access_logs, v_shipment_anomaly_score

8. Cara Menjalankan & Tautan Branch

Supabase / PostgreSQL lokal

```
psql -d anteraja -f schema.sql
psql -d anteraja -f seed.sql
psql -d anteraja -f queries.sql
```

Docker (PostGIS)

```
docker run -d --name anteraja-pg -e POSTGRES_PASSWORD=postgres \
-e POSTGRES_DB=anteraja -v "$PWD/docs/db":/db -p 55432:5432 postgis/postgis:16-3.4
docker exec anteraja-pg psql -U postgres -d anteraja -f /db/schema.sql
docker exec anteraja-pg psql -U postgres -d anteraja -f /db/seed.sql
```

Regenerasi ERD & Dokumen

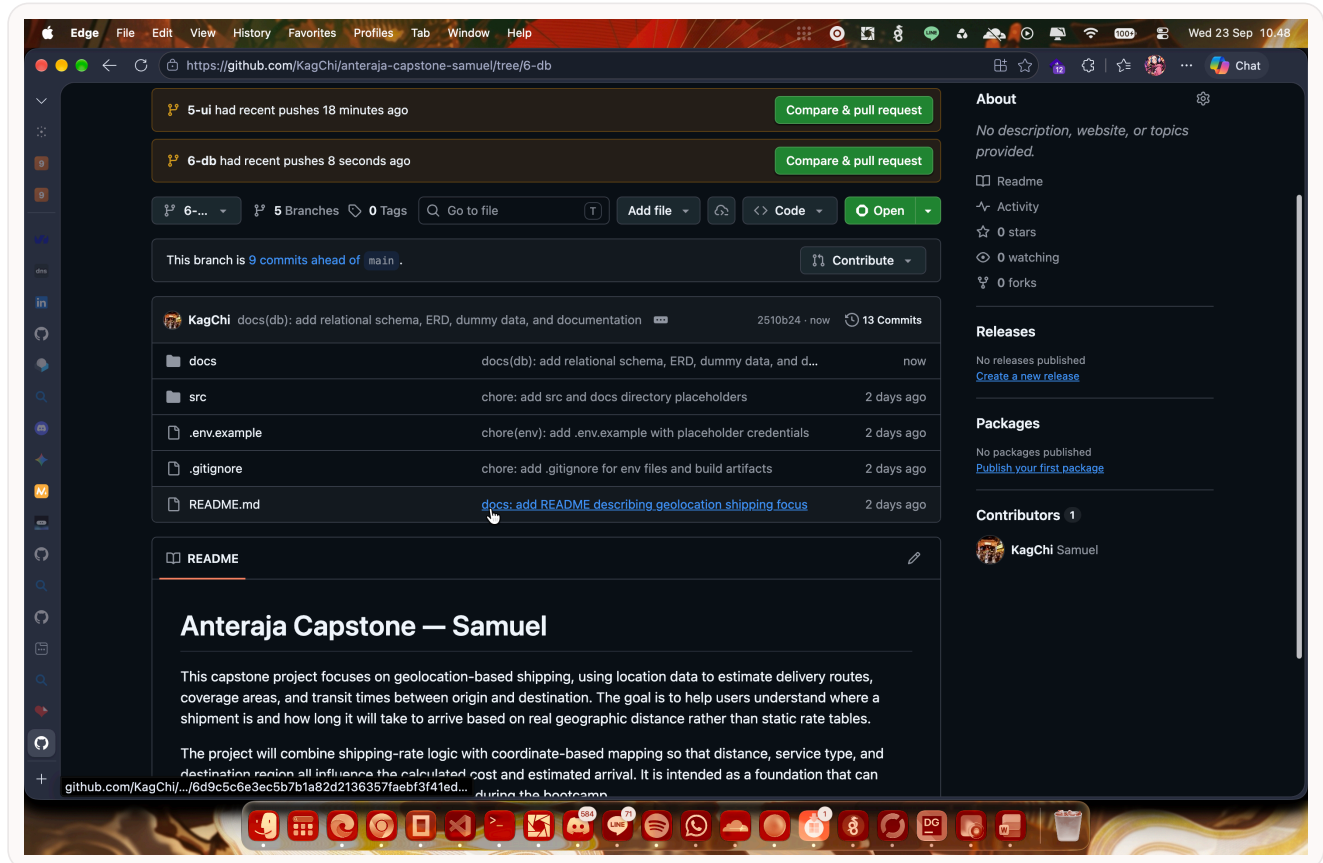
```
cd docs/db
dot -Tpng -Gdpi=150 erd.dot -o /tmp/erd.png && cwebp -q 90 /tmp/erd.png -o erd.webp
typst compile --font-path ../ui/fonts db-documentation.typ db-documentation.pdf
```

Tautan Branch

Seluruh berkas (schema.sql, seed.sql, queries.sql, erd.webp, README.md) berada pada branch **6-db**:
github.com/KagChi/anteraja-capstone-samuel/tree/6-db

9. Lampiran — Bukti Branch

Seluruh pekerjaan basis data ini dikerjakan pada branch Git **6-db** di repositori Anteraja Capstone. Tangkapan layar berikut menjadi bukti bahwa branch telah dibuat dan dipakai sebagai wadah pengumpulan berkas `schema.sql`, `seed.sql`, `queries.sql`, `erd.webp`, dan `README.md`.



Gambar: Bukti branch 6-db pada repositori Git Anteraja Capstone.